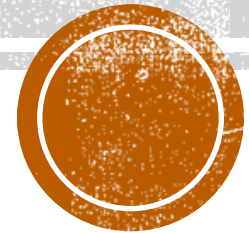




Module: Développer en Front-end

Redux Toolkit



Filière: Développement digital – option web full stack

Réalisé par: ELMAUDNI & YOUALA

Le problème avec Redux

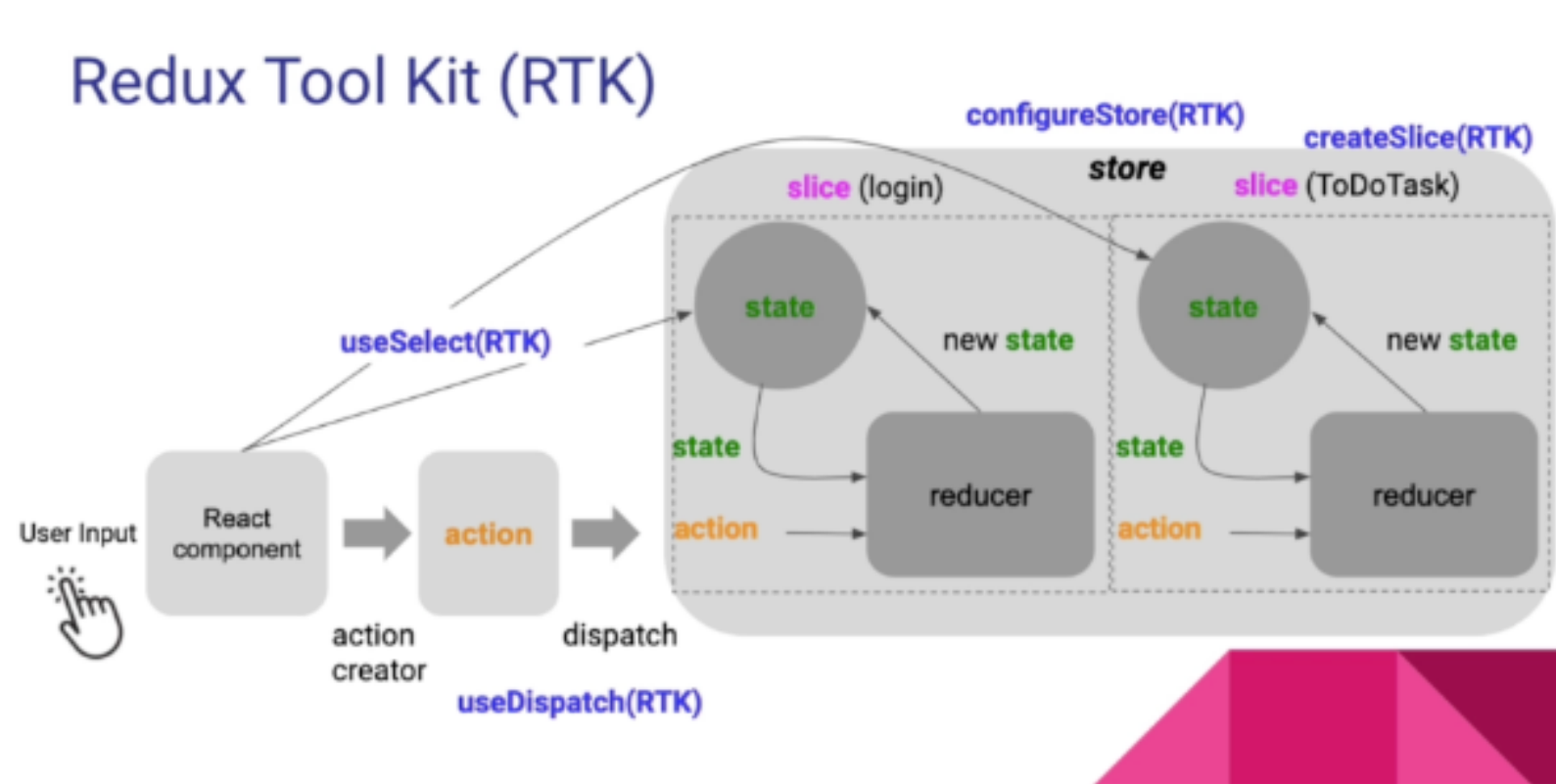
- Redux est un **choix populaire** pour la gestion d'état.
- React et Redux sont considérés comme **la meilleure combinaison pour gérer les états** dans les applications React à grande échelle. Cependant, avec le temps, la popularité de Redux **a chuté** pour les raisons suivantes :
 - Configurer un magasin Redux n'est pas simple.
 - Plusieurs packages sont nécessaires pour que Redux fonctionne avec React.
 - Redux nécessite trop de code passe-partout.
 - L'écriture d'actions et de réducteurs devient plus complexe et fastidieuse dans les applications volumineuses

Présentation de Redux Toolkit

- [Redux Toolkit](#) est l'approche standard officielle de React pour écrire la logique Redux.
- Il entoure le **noyau Redux** et contient **des packages** et **des fonctions** essentiels pour créer une application Redux.
- Redux Toolkit s'appuie sur les meilleures pratiques suggérées, simplifie la plupart des tâches Redux, évite les erreurs courantes et facilite l'écriture d'applications Redux.
- Le Redux Toolkit est livré avec plusieurs packages utiles installés, tels que **Immer**, **Redux-Thunk** et **Reselect**. Cela facilite la vie des développeurs React, en leur permettant de muter l'état directement avec moins de code (Immer gère l'immutabilité) et d'appliquer un middleware comme Thunk (qui gère les actions asynchrones) prêt à l'emploi. Il utilise également Reselect, une simple bibliothèque « sélecteur » pour Redux, pour simplifier les fonctions du réducteur et optimiser le code.
- tout comme Redux, Redux Toolkit n'est pas conçu uniquement pour **React**. Il peut être utilisé **avec n'importe quel autre framework** tel qu'Angular.

Concepts de base de Redux Toolkit (RTK)

- Un utilisateur interagit avec la **vue** pour initier un **événement d'action**;
- **Reducer** reçoit l'**action** de l'événement **dispatch** et **met à jour l'état** en conséquence;
- Lorsque **l'état est mis à jour** dans le **store**, les modifications sont modifiées via la méthode d'abonnement et **View met à jour son interface** en conséquence.



Concepts de base de Redux Toolkit (RTK)

- Le "**reducer**" (ou réducteur) dans Redux Toolkit est une fonction JavaScript qui spécifie comment l'état d'une application gérée par Redux **doit être modifié** en réponse à des **actions**.
- Redux Toolkit simplifie la création de réducteurs en utilisant une approche basée sur des "**slices**" (tranches), qui sont des portions de l'état global de l'application.
- Un réducteur dans Redux Toolkit est généralement créé à l'aide de la fonction **createSlice()**, qui prend en compte plusieurs arguments, dont **le nom du slice**, son **état initial**, et **un ensemble d'actions** (ou "**reducers**" à proprement parler) à appliquer à cet état.
- Un **store** (magasin):
 - Un store est un **conteneur d'état** qui contient **l'état de l'application**;
 - Il **gère** efficacement **tous les états** et **leurs mises à jour**;
 - Les applications Redux n'ont **qu'un seul store** pour gérer l'état.

Créer une application en utilisant Redux Toolkit

■ Etape 1:

- Pour démarrer avec les packages Redux Toolkit et React-Redux, vous pouvez exécuter la commande suivante sur une application React existante: **npm install @reduxjs/toolkit react-redux**
Pour créer une nouvelle application avec React: **npx create-react-app my-app --template redux**

Étape 2 : Créer, configurer et initialiser le store

- Créons un **store** pour contenir nos États. Créez un fichier **store.js** dans notre dossier **src** et ajoutez-y le code suivant.

```
// store.js
import { configureStore } from '@reduxjs/toolkit';
import xSlice from './xSlice';
import ySlice from './ySlice';

const store = configureStore({
  reducer: {
    x: xSlice.reducer,
    y: ySlice.reducer,
  },
});
export default store;
```

■ Étape 3 : Écrire des réducteurs et des actions

Écrivez maintenant quelques fonctions et actions de réduction pour notre store Redux.

Les réducteurs et les actions peuvent être écrits sous cette tranche (*slice*).

```
const xSlice = createSlice({
  name: 'x',
  initialState: {
    z: [], // z est maintenant un tableau vide
    t: // ... une autre propriété spécifique,
  },
  reducers: {
    addZ: (state, action) => {
      // Ajoute un élément à z en utilisant action.payload
      state.z.push(action.payload);
    },
    // ... d'autres réducteurs pour la tranche x
  },
});

export const { addZ } = xSlice.actions; export default xSlice;
```

```
const ySlice = createSlice({
  name: 'y',
  initialState: {
    someProperty: // ... une propriété spécifique
    pour le slice y
  },
  reducers: {
    // ... réducteurs pour la tranche y
  },
});
```

C'est en fait parce que Redux Toolkit utilise la bibliothèque Immer, qui permet d'écrire une logique de « mutation » dans les réducteurs. Cela conduit à un développement plus rapide et à des tests plus faciles.

- **Étape 4 : Distribuer des actions à partir du composant React UI**
- Comme nous l'avons appris précédemment, la vue déclenche l'envoi d'une action afin de mettre à jour un état. Dans Redux, utilisez **store.dispatch(action)** pour distribuer une action.
- Utilisons plutôt React-Redux pour utiliser le hook **useDispatch** pour distribuer des actions et **useSelector** pour lire les données du **store**.

```
import React, { useState } from 'react';
import { useSelector, useDispatch } from 'react-redux';
import { addZ } from './xSlice';

function MyComponent() {
  const xState = useSelector((state) => state.x);
  const dispatch = useDispatch();

  const [newValue, setNewValue] = useState("");

  const handleAddZ = () => {
    dispatch(addZ(newValue));
    setNewValue("");
  };
}
```

```
return (
  <div>
    <p>Valeur actuelle de z : {xState.z.join(', ')}</p>
    <input
      type="text"
      value={newValue}
      onChange={(e) => setNewValue(e.target.value)}
    />
    <button onClick={handleAddZ}>Ajouter à z</button>
  </div>
);
}

export default MyComponent;
```

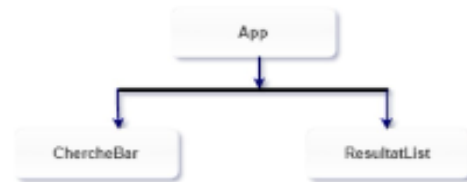
Exercice d'application

Exercice

On suppose que nous disposons de trois composants :

- App
- ChercherBar
- ResultatList

Les composants ChercherBar et ResultatList sont imbriqués dans le composant App



```
export const list = [  
  { nom: "banane", type: "fruit" },  
  { nom: "orange", type: "fruit" },  
  { nom: "pomme", type: "fruit" },  
  { nom: "raisins", type: "fruit" },  
  { nom: "carotte", type: "legume" },  
  { nom: "pomme de terre", type: "legume" },  
  { nom: "navet", type: "legume" },  
  { nom: "poivron", type: "legume" },  
]
```

Composant App

composant ChercherBar

Entrer le mot clé de recherche:

le type :LEGUME

composant ResultatList

- tomate
- carotte
- pomme de terre
- navet
- poivron

Exercice d'application

Le projet doit avoir la structure suivante:

Components
JS App.js
JS ChercherBar.js
JS ResultatList.js
JS data.js
JS ListSlice.js
JS store.js

Le flux de données en RTK pour cet exercice peut être présenté ainsi::

